

U.T.N.

Sintaxis y Semántica de los Lenguajes

Informe SSL TP1

“Autómatas en C: Informe
técnico”

Alumno: Kevin Ricardo Bernabe
Carnero

Comisión: K2002

Año lectivo: 2026

Índice

1.	Introducción	3
2.	Ejercicio 1	3
2.1.	Resolucion	3
2.2.	Definición de autómatas	4
2.3.	Implementación en C	5
2.4.	Ejecución del programa	6
3.	Ejercicio 2	7
3.1.	Resolución	7
3.2.	Implementación en C	7
3.3.	Ejecución del programa	7
4.	Ejercicio 3	8
4.1.	Resolución	8
4.2.	Definición de autómatas	8
4.3.	Implementación en C	8
4.4.	Ejecución del programa	9
5.	Conclusiones	10

Introducción

El presente trabajo práctico tiene como objetivo aplicar los conceptos de **Autómatas Finitos Determinísticos (AFD)** y expresiones regulares mediante su implementación en lenguaje C. A lo largo de los distintos ejercicios se desarrollan soluciones para el reconocimiento de constantes numéricas, la conversión de caracteres numéricos a enteros y la validación y evaluación de expresiones aritméticas simples.

Ejercicio 1

Dada una cadena que contenga **varios números que pueden ser decimales, octales o hexadecimales, con o sin signo** para el caso de los *decimales*, **separados** por el carácter '@', reconocer los tres grupos de constantes enteras, indicando si hubo un *error léxico*, en caso de ser correcto contar la cantidad de cada grupo.

Resolución:

Alfabeto Σ = { 0-9, A-F, a-f, x, X, +, -, @} }

Lenguajes a reconocer:

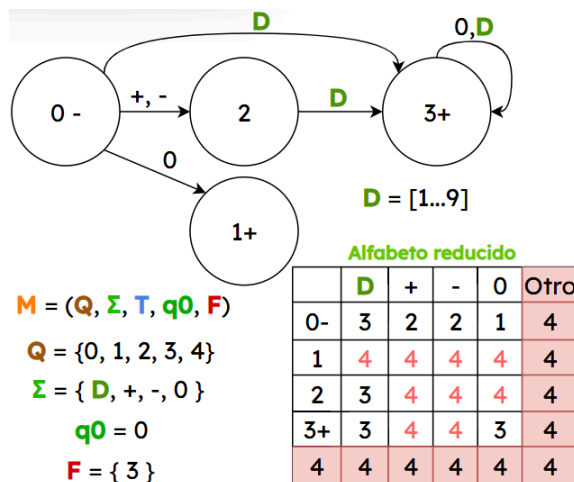
Decimal	Octal
ER Extendida: $0 \mid [+ -]?[1-9][0-9]^*$	ER Extendida: $0[0-7]^+$
Ejemplos válidos: +31, 12, 0, -4.	Ejemplos válidos: 01, 077, 0123
Hexadecimal	
ER Extendida: $0[xX][0-9A-Fa-f]^+$	
Ejemplos válidos: 0xFF, 0X1A, 0x10	

Posibles errores léxicos:

099	9 no es un dígito octal.
0x	Faltan dígitos hexadecimales.
+0xA	El signo sólo se permite para decimales.
-077	El signo no se permite para octales.
0xG1	G no es hexadecimal.
-0 o +0	No se toma por convención para no

Definición de autómatas

AFD para números decimales



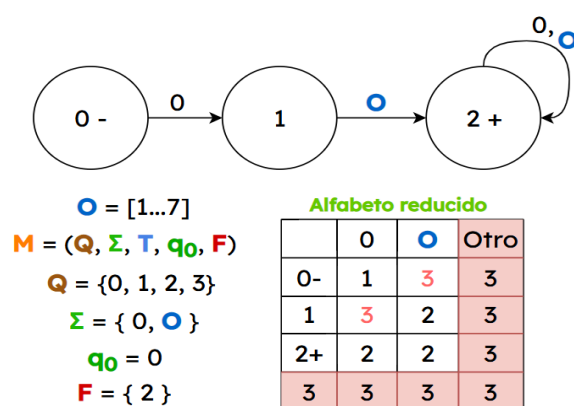
Autómata Finito Determinístico (AFD) para el reconocimiento de constantes enteras decimales con signo opcional.

El autómata acepta el **número 0** o **constantes enteras decimales sin ceros iniciales** con *signo opcional*, rechazando cualquier secuencia que no pertenezca al lenguaje definido.

La **Tabla de Transiciones (TT)** indica el **estado de llegada** del autómata para cada símbolo del alfabeto reducido, según el *estado actual*.

Se considera **D** a cualquier **dígito decimal** entre 1 y 9, separando el 0 para evitar ambigüedades con las **constantes octales**. El estado '4' representa el **estado de error** utilizado para completar el **AFD** y facilitar su implementación en C.

AFD para números Octales

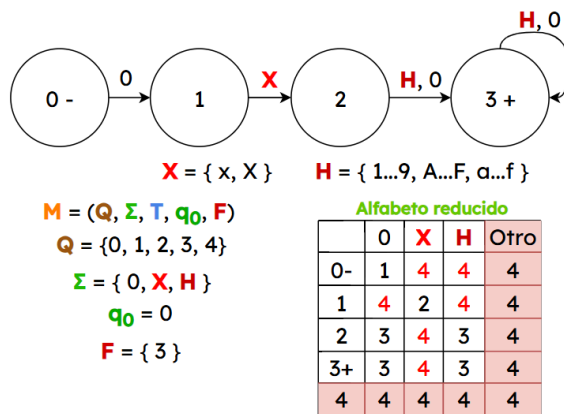


Autómata Finito Determinístico (AFD) para el reconocimiento de **constantes octales** que empiecen con '0'.

El autómata acepta **constantes octales** formadas por un '0' inicial seguido de *uno o más dígitos octales (0-7)*. En este trabajo se adopta esta **convención** para evitar la ambigüedad entre la **constante 0 decimal** y la **representación octal**.

Se considera **O** a cualquier **dígito octal** comprendido entre 1 y 7 (agregando al 0 por demás). Nuevamente, el estado '3' representa el **estado de error**, idem anterior.

AFD para números Hexadecimales



Autómata Finito Determinístico (AFD) para el reconocimiento de **constantes hexadecimales** que empiecen con '0' seguido de una 'X' o 'x'.

El autómata acepta **constantes hexadecimales** formadas por un '0' inicial, seguido de una 'x' o 'X' y uno o más **digitos hexadecimales** (1-9, A-F, a-f). En este trabajo se adopta esta **convención** para evitar ambigüedades durante el reconocimiento léxico.

Se considera **H** a cualquier **símbolo hexadecimal** comprendido entre 1 y 9 o las letras A-F (mayúsculas o minúsculas). El carácter 0 se representa por separado en el *alfabeto reducido* para distinguir el prefijo hexadecimal (0x o 0X) del resto de la constante.

Implementación en C

Los autómatas diseñados fueron **implementados en lenguaje C**, respetando la estructura definida por sus **tablas de transición**. Cada tipo de constante (**decimal**, **octal** y **hexadecimal**) se implementó en un *módulo independiente*, favoreciendo la organización y reutilización del código.

Para cada autómata se desarrollaron las siguientes funciones:

- Una función auxiliar para **traducir cada carácter de entrada** a una **columna de la tabla de transición** (alfabeto reducido).
- Una función encargada de **recorrer la cadena y simular el funcionamiento del autómata**.
- Una función de **verificación léxica** que comprueba que todos los caracteres de la cadena pertenezcan al alfabeto permitido.

De esta manera, la lógica de reconocimiento queda separada para cada tipo de constante, facilitando su mantenimiento y comprensión.

Organización del ejercicio

El código fue organizado siguiendo una estructura modular. Cada autómata posee su propio **archivo fuente (.c)** y su correspondiente cabecera (.h), mientras que el programa principal se encuentra en **main.c**, encargado de coordinar el análisis de las constantes.

Funcionamiento del programa

Durante esta implementación, la entrada del programa consiste en una cadena de prueba *definida* en el archivo **main.c**, compuesta por **constantes separadas** mediante el **carácter '@'**. Esta decisión se adoptó con el objetivo de facilitar las pruebas y la validación de los autómatas implementados.

Cada constante es *copiada* a un **buffer temporal** y posteriormente analizada por una función encargada de **determinar si corresponde a** una constante **decimal**, **octal**, **hexadecimal** o si representa un **error léxico**.

Durante el procesamiento también se contabiliza la cantidad de **constantes reconocidas** de **cada tipo** y la cantidad de **errores léxicos** detectados, mostrando un resumen al finalizar la ejecución.

La separación entre los distintos módulos facilita la incorporación de **nuevos tipos de constantes** sin modificar el resto del programa.

Compilación

Para automatizar la compilación del proyecto se incorporó un archivo **Makefile**. Desde la carpeta **ejercicio1** es posible compilar el programa mediante el comando **'make'** o ejecutarlo directamente con **'make run'**. Alternativamente puede generarse el ejecutable y ejecutar el mismo mediante la siguiente secuencia de comandos.

```
> gcc src/main.c src/decimal.c src/octal.c src/hexadecimal.c -Iinclude -o ejercicio1
> ./ejercicio1.exe
```

Ejecución del programa y ejemplo

Para verificar el correcto funcionamiento de los autómatas implementados, se ejecutó el programa utilizando una cadena de prueba compuesta por constantes **decimales**, **octales** y **hexadecimales**.

La siguiente figura muestra un ejemplo de ejecución del programa con la **siguiente cadena**:

-123@056@0x1AF@08@0xG@0@+456

```
Ejercicio 1 - Cadenas y tipos de constantes
-----
Cadenas leídas:
-123 -> Constante decimal
056 -> Constante octal
0x1AF -> Constante hexadecimal
08 -> Error lexico
0xG -> Error lexico
0 -> Constante decimal
+456 -> Constante decimal
-----
RESUMEN de constantes leídas:
-----
Constantes decimales: 3
Constantes octales: 1
Constantes hexadecimales: 1
Se detectaron errores 2 lexicos.
```

La **documentación técnica completa del ejercicio**, así como las instrucciones de utilización del programa, se presentan en el **manual correspondiente** adjunto al repositorio y la carpeta del ejercicio correspondiente.

Ejercicio 2

Debe realizar una función que reciba un carácter numérico y retorne un número entero.

Resolución:

Implementación en C

La solución consiste en una función independiente que **recibe un carácter** y determina si corresponde a un **dígito decimal**. En caso afirmativo, devuelve su valor entero utilizando la diferencia entre el **código ASCII** del **carácter recibido** y el carácter **'0'**. Si el carácter no pertenece al conjunto de los dígitos decimales, la función retorna **-1**.

Compilación

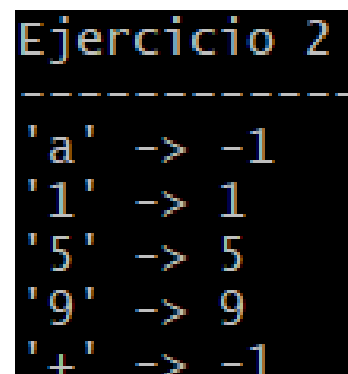
Para automatizar la compilación del proyecto se incorporó un archivo **Makefile**. Desde la carpeta **ejercicio2** es posible compilar el programa mediante el comando **'make'** o ejecutarlo directamente con **'make run'**. Alternativamente puede generarse el ejecutable y ejecutar el mismo mediante la siguiente secuencia de comandos.

```
> gcc src/main.c src/conversion.c -Iinclude -o ejercicio2
> ./ejercicio2.exe
```

Ejecución del programa y ejemplo

Para comprobar el correcto funcionamiento de la función se realizaron pruebas utilizando distintos *caracteres de entrada*, incluyendo **dígitos decimales válidos** y **caracteres que no pertenecen al lenguaje**.

La figura siguiente muestra algunos de los casos evaluados, donde puede observarse que la función devuelve el **valor entero** correspondiente para los dígitos (**'1'**, **'5'**, **'9'**) y retorna -1 cuando la entrada no representa un carácter numérico válido, como ocurre con **'a'** o **'+'**.



Ejercicio 2

'a'	->	-1
'1'	->	1
'5'	->	5
'9'	->	9
'+'	->	-1

La **documentación técnica completa del ejercicio**, así como las instrucciones de utilización del programa, se presentan en el **manual correspondiente** adjunto al repositorio y la carpeta del ejercicio correspondiente.

Ejercicio 3

Ingresar una **cadena** que represente una *operación simple* con enteros decimales y obtener su resultado, se debe operar con +, - y *. Ejemplo = $3+4*7+3-5 = 29$.

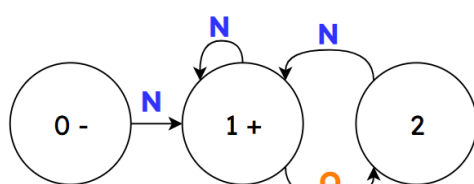
Considerando que el alfabeto son los números decimales, signos +, - y * valide con un autómata previo a la operación que dicha cadena que representa la operación pertenezca al lenguaje.

Resolución:

Alfabeto $\Sigma = \{ 0-9, +, -, * \}$

Lenguaje a reconocer: ER Extendida: $[0-9]^+([+\backslash-\backslash*][0-9]^+)^*$

Definición de autómatas



$N = [0-9]$ $O = \{ +, -, * \}$

$M = (Q, \Sigma, T, q_0, F)$

$Q = \{0, 1, 2, 3\}$

$\Sigma = \{ N, O \}$

$q_0 = 0$

$F = \{ 1 \}$

Alfabeto reducido

	N	O	Otro
0-	1	3	3
1+	1	2	3
2	1	3	3
3	3	3	3

Autómata Finito Determinístico (AFD) para el reconocimiento de **expresiones aritméticas simples**.

El autómata acepta expresiones formadas por **números enteros decimales** y los operadores +, - y *, verificando que la expresión comience y finalice con un **número** y que los operadores aparezcan únicamente entre **operandos válidos**.

La **Tabla de Transiciones (TT)** indica el **estado de llegada** para cada símbolo del alfabeto reducido según el **estado actual**.

Se considera **N** al conjunto de los **dígitos decimales** (0-9), **O** al conjunto de **operadores** (+, -, *) y **Otro** a cualquier carácter que no pertenezca al alfabeto. El estado '3' representa el **estado de error**, utilizado para completar el **AFD** y facilitar su implementación en **C**.

Implementación en C

Los componentes desarrollados para este ejercicio fueron **implementados en lenguaje C**, separando la **validación sintáctica de la expresión** de su **evaluación aritmética**.

La validación se realiza mediante un **Autómata Finito Determinístico (AFD)**, encargado de verificar que la **expresión pertenezca al lenguaje definido**. Una vez validada la entrada, el módulo **evaluador** procesa la expresión respetando la **precedencia de los operadores**, otorgando prioridad a la **multiplicación** sobre la **suma** y la **resta**. Para ello, divide la expresión en **términos**, resuelve primero las **multiplicaciones** y posteriormente aplica las operaciones de suma y resta hasta obtener el resultado final.

Organización del ejercicio

El código fue organizado de forma modular.

- **automata.c / automata.h**: implementan el **AFD** encargado de verificar que la expresión sea *sintácticamente válida*.
- **evaluador.c / evaluador.h**: contienen las funciones responsables de **leer números, resolver operaciones y evaluar la expresión completa**.
- **main.c**: coordina el funcionamiento del programa, validando primero la expresión y, si pertenece al lenguaje, obteniendo e imprimiendo el resultado final de la operación.

Funcionamiento del programa

El programa recibe una expresión aritmética (predefinida en **main.c**) compuesta por números enteros y operadores **+**, **-** y *****.

En primer lugar, la expresión es analizada por el **AFD** definido para comprobar que cumpla la sintaxis definida por el lenguaje. Si la **validación** resulta satisfactoria, la expresión es enviada al **evaluador**, el cual procesa cada *término* respetando la *precedencia de la multiplicación* y calcula el resultado final de la operación.

Compilación

Para automatizar la compilación se incorporó un archivo **Makefile**. Desde la carpeta **ejercicio3** es posible compilar el programa mediante el comando **'make'** o ejecutarlo directamente con **'make run'**.

Alternativamente, el programa puede compilarse y ejecutarse manualmente mediante la siguiente línea de comandos (todo dentro de la carpeta **ejercicio3**):

```
> gcc src/main.c src/automata.c src/evaluador.c -Iinclude -o ejercicio3  
> ./ejercicio3.exe
```

Ejecución del programa y ejemplo

Para verificar el correcto funcionamiento del programa se realizaron pruebas utilizando distintas expresiones aritméticas válidas e inválidas.

La siguiente figura muestra un **ejemplo** de ejecución utilizando la expresión:

3+4*7+3-5

Obteniéndose como resultado: **29**

```
Ejercicio 3 - Resultado de expresion  
-----  
Expresion a evaluar: 3+4*7+3-5  
Expresion valida  
Resultado: 29
```

La **documentación técnica completa del ejercicio**, así como las instrucciones de utilización del programa, se presentan en el **manual correspondiente** adjunto al repositorio y la carpeta del ejercicio correspondiente.

Conclusiones

Durante el desarrollo del trabajo práctico se implementaron distintos **autómatas finitos determinísticos** para el reconocimiento de *lenguajes específicos*, complementándolos con módulos en lenguaje C para su simulación y evaluación. La organización modular del código permitió separar claramente las tareas de validación léxica, procesamiento y evaluación de expresiones, facilitando el mantenimiento y la reutilización de los distintos componentes desarrollados.

Repositorio del proyecto:

<https://github.com/bernabekevin4/SSL---TP1-Automatas---Kevin-Bernabe.git>